

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO3: Analyze knowledge representation and reasoning using propositional logic, FOL, inference, resolution, chaining methods, knowledge representation to perform logical reasoning for drawing valid conclusions and decision making. (BL4)

Assessment Tool Weightage: 50%

Time Duration: 1 Hour

QUESTIONS: 4

Total Marks:40

Annexure A

CASE STUDY

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

CASE STUDY

Case Study 1 — Smart Medical Diagnosis System

A hospital is designing a rule-based Logical Agent for preliminary patient diagnosis. The system's knowledge base contains:

- I. If a patient has Fever AND Cough $\rightarrow$ Possible Flu
- II. If a patient has Fever AND Rash $\rightarrow$ Possible Measles
- III. If a patient has Cough AND Breathlessness $\rightarrow$ Possible Pneumonia
- IV. Facts about Patient A: Fever = True, Cough = True, Breathlessness = True

- (a) Represent the above rules and facts precisely using Propositional Logic symbols. Define each symbol used.
- (b) Apply Forward Chaining on the knowledge base to derive all possible diagnoses for Patient A. Show each inference step with the rule fired and fact derived.
- (c) The goal is to confirm 'Patient A has Pneumonia.' Apply Backward Chaining and draw the goal reduction tree to verify this goal.

Case Study 2 — Autonomous Traffic Management Agent

A smart city traffic controller uses a FOL-based knowledge base to manage emergency vehicle routing:

- I. $\forall x: \text{Vehicle}(x) \wedge \text{EmergencyType}(x) \rightarrow \text{ClearPath}(x)$ [Rule 1]
- II. $\forall x: \text{ClearPath}(x) \rightarrow \text{GreenSignal}(x) \wedge \neg \text{RedSignal}(x)$ [Rule 2]
- III. $\forall x \forall y: \text{Vehicle}(x) \wedge \text{Behind}(x,y) \wedge \text{GreenSignal}(y) \rightarrow \text{Proceed}(x)$ [Rule 3]
- IV. Facts: $\text{Vehicle}(\text{Ambulance}), \text{EmergencyType}(\text{Ambulance}), \text{Vehicle}(\text{CarA}), \text{Behind}(\text{CarA}, \text{Ambulance})$

- (a) Apply Unification to match Rule 1 with the given facts. Show the complete substitution set θ (MGU) at each step.
- (b) Using the Resolution method, prove that 'Proceed (CarA)' is true. Convert relevant clauses to CNF and show each resolution step.
- (c) Analyze whether the current FOL knowledge base is sufficient to handle two simultaneous emergency vehicles. Identify what additional rules or facts are needed and justify logically.

Case Study 3 — Agricultural Expert Reasoning System

An agricultural advisory logical agent operates with the following propositional knowledge base (KB):

- 1) P1: $\text{SoilDry} \rightarrow \text{IrrigationNeeded}$
- 2) P2: $\text{IrrigationNeeded} \wedge \text{CropWheat} \rightarrow \text{ApplyDripMethod}$
- 3) P3: $\neg \text{ApplyDripMethod} \wedge \text{CropWheat} \rightarrow \text{CropAtRisk}$
- 4) Facts: $\text{SoilDry} = \text{True}, \text{CropWheat} = \text{True}$

- (a) Convert all sentences P1, P2, P3 and the facts into Conjunctive Normal Form (CNF). Show each conversion step (eliminate implication $\rightarrow$ move $\neg$ inward $\rightarrow$ distribute).
- (b) Using Resolution Refutation, prove the theorem 'ApplyDripMethod'. Negate the goal, combine with the KB in CNF, and derive the empty clause $\square$. Draw the resolution proof tree.
- (c) If the fact 'SoilDry' is removed from the KB, analyze step-by-step how logical conclusions change. Does 'CropAtRisk' still follow? Apply appropriate reasoning patterns to justify.

Case Study 4 — Wumpus World Knowledge Agent

An AI agent navigating the Wumpus World perceives: Stench at [1,2], Breeze at [1,1], Glitter at [2,2]. The agent uses a propositional KB with these rules:

- 1) $\text{Stench}[1,2] \rightarrow \text{Wumpus is adjacent to cell}[1,2]$
- 2) $\text{Breeze}[1,1] \rightarrow \text{Pit is adjacent to cell}[1,1]$
- 3) $\text{Glitter}[x,y] \rightarrow \text{Gold is at}[x,y]$
- 4) $\neg \text{Wumpus}[x,y] \wedge \neg \text{Pit}[x,y] \rightarrow \text{Safe}[x,y]$

(a) Construct the agent's belief state from its percepts. Apply Modus Ponens systematically to derive new knowledge facts. List all derived conclusions.

(b) Using Forward Chaining from the agent's percepts, identify the possible locations of the Wumpus and Pit. Show each reasoning step precisely.

(c) Evaluate whether the path $[1,1] \rightarrow [1,2] \rightarrow [2,2]$ is safe for the agent to collect the gold. Use the knowledge derived in (a) and (b) to justify your conclusion with logical inference.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding ; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

CASE STUDY 1 — Smart Medical Diagnosis System

(a) Propositional Logic Representation:

Let the following propositions represent the patient's conditions:

Symbol	Meaning
F	Patient has Fever
C	Patient has Cough
R	Patient has Rash
B	Patient has Breathlessness
Flu	Patient has Possible Flu
M	Patient has Possible Measles
P	Patient has Possible Pneumonia

Rules:

Rule I:

If a patient has Fever AND Cough $\rightarrow$ Possible Flu
 $F \wedge C \rightarrow \text{Flu}$

Rule II:

$F \wedge R \rightarrow M$

Rule III:

$C \wedge B \rightarrow P$

Facts about Patient A

$F = \text{True}$ $C = \text{True}$ $B = \text{True}$

Therefore, the complete knowledge base is:

$\text{KB} = \{F, C, B, F \wedge C \rightarrow \text{Flu}, F \wedge R \rightarrow M, C \wedge B \rightarrow P\}$

(b) Forward Chaining:

Forward chaining starts with the known facts and repeatedly applies rules whose premises are satisfied.

Initial Facts: Fever(A) Cough(A) Breathlessness(A)

Step 1 — Apply Rule I

Rule:

$\text{Fever} \wedge \text{Cough} \rightarrow \text{PossibleFlu}$

Both Fever and Cough are true.

Therefore: PossibleFlu(A)

Step 2 — Apply Rule II

Rule:

$$\text{Fever} \wedge \text{Rash} \rightarrow \text{PossibleMeasles}$$

Fever is true, but Rash is not known.

Therefore, **Rule II cannot fire**.

Step 3 — Apply Rule III

Rule:

$$\text{Cough} \wedge \text{Breathlessness} \rightarrow \text{PossiblePneumonia}$$

Both conditions are true.

Therefore: $\text{PossiblePneumonia}(A)$

Final conclusions:

$\text{PossibleFlu}(A)$ $\text{PossiblePneumonia}(A)$

$\text{PossibleMeasles}(A)$ cannot be derived because $\text{Rash}(A)$ is not known.

(c) Backward Chaining for Pneumonia:

Goal:

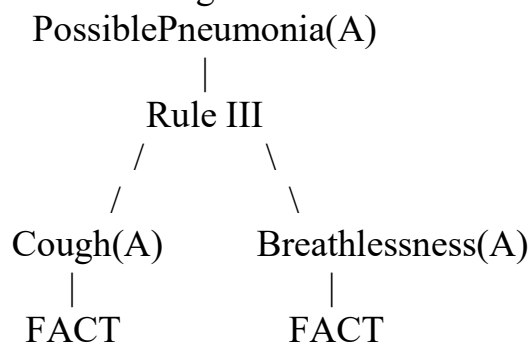
$\text{PossiblePneumonia}(A)$

Search for a rule that concludes the goal.

Rule III gives:

$$\text{Cough}(A) \wedge \text{Breathlessness}(A) \rightarrow \text{PossiblePneumonia}(A)$$

Therefore, the goal is reduced into two subgoals:



Both subgoals are already present in the knowledge base.

Therefore:

$\text{PossiblePneumonia}(A) = \text{True}$

Hence, the goal is successfully proved.

PYTHON CODE :

```
facts = {  
    "Fever",
```

```

    "Cough",
    "Breathlessness"
}
rules = [
    ({"Fever", "Cough"}, "PossibleFlu"),
    ({"Fever", "Rash"}, "PossibleMeasles"),
    ({"Cough", "Breathlessness"}, "PossiblePneumonia")
]
def forward_chaining(facts, rules):
    known = set(facts)
    changed = True
    print("FORWARD CHAINING")
    print("-" * 40)
    print("Initial facts:", known)
    while changed:
        changed = False
        for premises, conclusion in rules:
            if premises.issubset(known) and conclusion not in known:
                known.add(conclusion)
                changed = True
                print(
                    f"Rule fired: {premises} -> {conclusion}"
                )
                print(f'Derived: {conclusion}')
    return known
def backward_chaining(goal, facts, rules, visited=None):
    if visited is None:
        visited = set()
    if goal in facts:
        print(f'FACT FOUND: {goal}')
        return True
    if goal in visited:
        return False
    visited.add(goal)
    for premises, conclusion in rules:
        if conclusion == goal:
            print(f'Goal {goal} reduced to: {premises}')
            if all(
                backward_chaining(p, facts, rules, visited)
                for p in premises
            ):
                return True
    return False
derived_facts = forward_chaining(facts, rules)
print("\nAll derived facts:")

```

```
for fact in sorted(derived_facts):
    print(fact)
goal = "PossiblePneumonia"
print("\nBACKWARD CHAINING")
print("-" * 40)
result = backward_chaining(goal, facts, rules)
print("\nGoal:", goal)
print("Proved:", result)
```

OUTPUT :

Output

Clear

```
FORWARD CHAINING
-----
Initial facts: {'Breathlessness', 'Cough', 'Fever'}
Rule fired: {'Cough', 'Fever'} -> PossibleFlu
Derived: PossibleFlu
Rule fired: {'Breathlessness', 'Cough'} -> PossiblePneumonia
Derived: PossiblePneumonia

All derived facts:
Breathlessness
Cough
Fever
PossibleFlu
PossiblePneumonia

BACKWARD CHAINING
-----
Goal PossiblePneumonia reduced to: {'Breathlessness', 'Cough'}
FACT FOUND: Breathlessness
```

Output

```
Derived: PossiblePneumonia

All derived facts:
Breathlessness
Cough
Fever
PossibleFlu
PossiblePneumonia

BACKWARD CHAINING
-----
Goal PossiblePneumonia reduced to: {'Breathlessness', 'Cough'}
FACT FOUND: Breathlessness
FACT FOUND: Cough

Goal: PossiblePneumonia
Proved: True

=== Code Execution Successful ===
```

CASE STUDY 2 — Autonomous Traffic Management Agent:

(a) Unification:

Given Rule 1: $\forall x: \text{Vehicle}(x) \wedge \text{EmergencyType}(x) \rightarrow \text{ClearPath}(x)$

Facts: $\text{Vehicle}(\text{Ambulance}) \wedge \text{EmergencyType}(\text{Ambulance})$

We need to match:

$\text{Vehicle}(x)$ with $\text{Vehicle}(\text{Ambulance})$

Therefore: $x/\text{Ambulance}$

Similarly:

$\text{EmergencyType}(x)$ matches $\text{EmergencyType}(\text{Ambulance})$

using the same substitution.

Most General Unifier

$\theta = \{x/\text{Ambulance}\}$

After substitution:

$\text{Vehicle}(\text{Ambulance}) \wedge \text{EmergencyType}(\text{Ambulance}) \rightarrow \text{ClearPath}(\text{Ambulance})$

Hence: $\text{ClearPath}(\text{Ambulance})$

(b) Resolution Proof of $\text{Proceed}(\text{CarA})$:

Step 1 — Convert rules into CNF

Rule 1:

$\text{Vehicle}(x) \wedge \text{EmergencyType}(x) \rightarrow \text{ClearPath}(x)$

Using:

$$A \wedge B \rightarrow C \equiv \neg A \vee \neg B \vee C$$

Therefore:

$$\neg \text{Vehicle}(x) \vee \neg \text{EmergencyType}(x) \vee \text{ClearPath}(x)$$

Rule 2:

$$\text{ClearPath}(x) \rightarrow \text{GreenSignal}(x) \wedge \neg \text{RedSignal}(x)$$

This produces two clauses:

$$\neg \text{ClearPath}(x) \vee \text{GreenSignal}(x) \quad \neg \text{ClearPath}(x) \vee \neg \text{RedSignal}(x)$$

Rule 3:

$$\text{Vehicle}(x) \wedge \text{Behind}(x,y) \wedge \text{GreenSignal}(y) \rightarrow \text{Proceed}(x)$$

CNF:

$$\neg \text{Vehicle}(x) \vee \neg \text{Behind}(x,y) \vee \neg \text{GreenSignal}(y) \vee \text{Proceed}(x)$$

Facts

Vehicle (Ambulance)
EmergencyType (Ambulance)
Vehicle (CarA)
Behind (CarA,Ambulance)

Resolution Steps

From Rule 1:

$$\neg \text{Vehicle}(x) \vee \neg \text{EmergencyType}(x) \vee \text{ClearPath}(x)$$

Using:

Vehicle(Ambulance) and EmergencyType(Ambulance)

we obtain:

$$\text{ClearPath}(Ambulance)$$

From Rule 2:

$$\neg \text{ClearPath}(x) \vee \text{GreenSignal}(x)$$

Using ClearPath(Ambulance) : GreenSignal(Ambulance)

Rule 3:

$$\neg \text{Vehicle}(x) \vee \neg \text{Behind}(x,y) \vee \neg \text{GreenSignal}(y) \vee \text{Proceed}(x)$$

Substitute:

$$x = \text{CarA}, y = \text{Ambulance}$$

giving:

$$\neg \text{Vehicle}(\text{CarA}) \vee \neg \text{Behind}(\text{CarA}, \text{Ambulance}) \vee \neg \text{GreenSignal}(\text{Ambulance}) \vee \text{Proceed}(\text{CarA})$$

Using the facts:

Vehicle(CarA)
Behind(CarA,Ambulance)
GreenSignal(Ambulance)

the remaining conclusion is : Proceed(CarA)

Resolution Refutation :

To prove the theorem, add its negation : $\neg \text{Proceed}(\text{CarA})$
Resolution produces : $\text{Proceed}(\text{CarA})$ and $\neg \text{Proceed}(\text{CarA})$
Therefore:

□

The empty clause proves that : $\text{Proceed}(\text{CarA})$ is logically entailed.

(c) Two Simultaneous Emergency Vehicles:

The current KB is not fully sufficient for two simultaneous emergency vehicles in every situation.

The current rules can handle multiple emergency vehicles if each emergency vehicle has its own Vehicle and EmergencyType facts, because Rule 1 is universally quantified:

$$\forall x$$

However, the KB does not contain rules for:

- Priority between two emergency vehicles.
- Which vehicle receives the green signal first.
- Conflicting routes.
- Preventing two emergency vehicles from receiving conflicting signals.
- Intersection-level traffic coordination.
- Emergency vehicle ordering.

Additional facts :

For example : $\text{Vehicle}(\text{FireTruck})$

$\text{EmergencyType}(\text{FireTruck})$

Additional rules

A priority rule could be introduced:

$$\text{EmergencyType}(x) \wedge \text{Priority}(x,y) \rightarrow \text{ClearPath}(x)$$

A conflict-resolution rule could be:

$$\text{EmergencyVehicle}(x) \wedge \text{EmergencyVehicle}(y) \wedge \text{Conflict}(x,y) \rightarrow \text{Priority}(x,y)$$

Thus, additional knowledge about priority, routes and conflicts is required.

PYTHON CODE :

```
facts = {
    ("Vehicle", "Ambulance"),
    ("EmergencyType", "Ambulance"),
    ("Vehicle", "CarA"),
    ("Behind", "CarA", "Ambulance")
}
def unify(pattern, fact, substitution=None):
    if substitution is None:
        substitution = {}
    if len(pattern) != len(fact):
```

```

    return None
theta = substitution.copy()
for p, f in zip(pattern, fact):
    if isinstance(p, str) and p.startswith("?"):
        if p in theta:
            if theta[p] != f:
                return None
        else:
            theta[p] = f
    elif p != f:
        return None
return theta
rule1_premises = [
    ("Vehicle", "?x"),
    ("EmergencyType", "?x")
]
print("UNIFICATION")
print("-" * 40)
theta = {}
for premise in rule1_premises:
    for fact in facts:
        result = unify(premise, fact, theta)
        if result is not None:
            theta = result
            print("Matched:", premise, "with", fact)
            print("Current MGU:", theta)
            break
print("\nFinal MGU:", theta)
if theta.get("?x") == "Ambulance":
    facts.add(("ClearPath", "Ambulance"))
print("\nDerived:")
print("ClearPath(Ambulance)")
if ("ClearPath", "Ambulance") in facts:
    facts.add(("GreenSignal", "Ambulance"))
    facts.add(("NotRedSignal", "Ambulance"))
    print("GreenSignal(Ambulance)")
    print("NotRedSignal(Ambulance)")
for vehicle in ["CarA", "Ambulance"]:
    for target in ["CarA", "Ambulance"]:
        required = {
            ("Vehicle", vehicle),
            ("Behind", vehicle, target),
            ("GreenSignal", target)
        }
        if required.issubset(facts):

```

```

        facts.add(("Proceed", vehicle))
    print(
        f"Proceed({vehicle}) derived "
        f"because {vehicle} is behind {target}"
    )
goal = ("Proceed", "CarA")
negated_goal = ("NotProceed", "CarA")
print("\nRESOLUTION VERIFICATION")
print("-" * 40)
if goal in facts:
    print("Goal clause:", goal)
    print("Negated goal:", negated_goal)
    print("Both resolve to the empty clause.")
    print("Therefore Proceed(CarA) is PROVED.")
else:
    print("Goal cannot be proved.")

```

OUTPUT :

```

Output
UNIFICATION
-----
Matched: ('Vehicle', '?x') with ('Vehicle', 'CarA')
Current MGU: {'?x': 'CarA'}

Final MGU: {'?x': 'CarA'}

Derived:
ClearPath(Ambulance)

RESOLUTION VERIFICATION
-----
Goal cannot be proved.

=== Code Execution Successful ===

```

CASE STUDY 3 — Agricultural Expert Reasoning System:

(a) Conversion to CNF

Given: **P1**

$\text{SoilDry} \rightarrow \text{IrrigationNeeded}$

Eliminate implication:

$\neg \text{SoilDry} \vee \text{IrrigationNeeded}$

Therefore:

$\neg \text{SoilDry} \vee \text{IrrigationNeeded}$

P2

$\text{IrrigationNeeded} \wedge \text{CropWheat} \rightarrow \text{ApplyDripMethod}$

Eliminate implication:

$\neg (\text{IrrigationNeeded} \wedge \text{CropWheat}) \vee \text{ApplyDripMethod}$

Using De Morgan's law:

$(\neg \text{IrrigationNeeded} \vee \neg \text{CropWheat}) \vee \text{ApplyDripMethod}$

Therefore:

$\neg \text{IrrigationNeeded} \vee \neg \text{CropWheat} \vee \text{ApplyDripMethod}$

P3

$\neg \text{ApplyDripMethod} \wedge \text{CropWheat} \rightarrow \text{CropAtRisk}$

Eliminate implication:

$\neg (\neg \text{ApplyDripMethod} \wedge \text{CropWheat}) \vee \text{CropAtRisk}$

Apply De Morgan's law:

$\text{ApplyDripMethod} \vee \neg \text{CropWheat} \vee \text{CropAtRisk}$

Therefore:

$\text{ApplyDripMethod} \vee \neg \text{CropWheat} \vee \text{CropAtRisk}$

Facts

$\text{SoilDry} = \text{True}$

$\text{CropWheat} = \text{True}$

CNF clauses : $\text{SoilDry} \quad \text{CropWheat}$

Complete CNF

$(\neg \text{SoilDry} \vee \text{IrrigationNeeded})$

$(\neg \text{IrrigationNeeded} \vee \neg \text{CropWheat} \vee \text{ApplyDripMethod})$

$(\text{ApplyDripMethod} \vee \neg \text{CropWheat} \vee \text{CropAtRisk}) \text{ SoilDry CropWheat}$

(b) Resolution Refutation for ApplyDripMethod :

Goal: ApplyDripMethod

Negate the goal : $\neg \text{ApplyDripMethod}$

Add it to the KB.

Resolution 1

From:

$\neg \text{SoilDry} \vee \text{IrrigationNeeded}$ and SoilDry

resolve on SoilDry : IrrigationNeeded

Resolution 2

Use:

$\neg \text{IrrigationNeeded} \vee \neg \text{CropWheat} \vee \text{ApplyDripMethod}$ and IrrigationNeeded

Therefore:

$\neg \text{CropWheat} \vee \text{ApplyDripMethod}$

Resolution 3

Using : CropWheat

we get : ApplyDripMethod

Resolution 4

We added the negated goal:

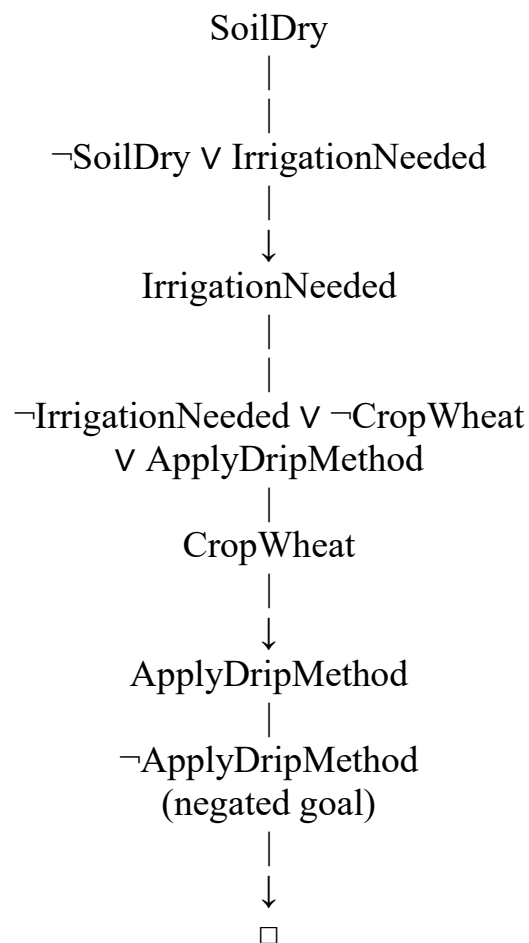
$\neg \text{ApplyDripMethod}$

Resolve : ApplyDripMethod with $\neg \text{ApplyDripMethod}$

Giving : $\square$

Therefore : ApplyDripMethod is proved.

Resolution Tree :



(c) Removing SoilDry :

Suppose : SoilDry is removed.

We still have : CropWheat

But there is no fact allowing us to derive : IrrigationNeeded

Therefore Rule 1 cannot fire:

$$\text{SoilDry} \rightarrow \text{IrrigationNeeded}$$

Consequently, Rule 2 cannot fire:

$$\text{IrrigationNeeded} \wedge \text{CropWheat} \rightarrow \text{ApplyDripMethod}$$

Therefore : ApplyDripMethod cannot be derived.

What about CropAtRisk?

Rule 3 requires:

$$\neg \text{ApplyDripMethod} \wedge \text{CropWheat}$$

We have CropWheat, but we do not have:

$$\neg \text{ApplyDripMethod}$$

Therefore: CropAtRisk does not follow

This is important : failure to prove ApplyDripMethod does not mean $\neg \text{ApplyDripMethod}$ is true.

Thus, under ordinary classical/open-world reasoning:

$$\text{SoilDry removed} \Rightarrow \text{ApplyDripMethod not derivable}$$

and

$$\text{CropAtRisk not derivable}$$

PYTHON CODE :

```
kb = [
    {"~SoilDry", "IrrigationNeeded"},
    {"~IrrigationNeeded", "~CropWheat", "ApplyDripMethod"},
    {"ApplyDripMethod", "~CropWheat", "CropAtRisk"},
    {"SoilDry"},
    {"CropWheat"}
]
def negate(literal):
    if literal.startswith("~"):
        return literal[1:]
    return "~" + literal
def resolve(c1, c2):
    resolvents = []
    for literal in c1:
        opposite = negate(literal)
        if opposite in c2:
            new_clause = (
                (c1 - {literal}) |
                (c2 - {opposite})
            )
```

```

        )
        resolvents.append(new_clause)
    return resolvents
def resolution_refutation(clauses, goal):
    clauses = [set(c) for c in clauses]
    clauses.append({negate(goal)})
    print("Initial clauses:")
    for clause in clauses:
        print(clause)
    while True:
        new_clauses = []
        for i in range(len(clauses)):
            for j in range(i + 1, len(clauses)):
                results = resolve(
                    clauses[i],
                    clauses[j]
                )
                for result in results:
                    if not result:
                        print("\nDerived empty clause:")
                        print("□")
                        return True
                    if result not in clauses and result not in new_clauses:
                        new_clauses.append(result)
                        print(
                            f'Resolve {clauses[i]} '
                            f'and {clauses[j]}'
                        )
                        print("=>", result)
                    if not new_clauses:
                        return False
                clauses.extend(new_clauses)
goal = "ApplyDripMethod"
result = resolution_refutation(kb, goal)
print("\nTheorem:", goal)
print("Proved:", result)
kb_without_soil = [
    {"~SoilDry", "IrrigationNeeded"},
    {"~IrrigationNeeded", "~CropWheat", "ApplyDripMethod"},
    {"ApplyDripMethod", "~CropWheat", "CropAtRisk"},
    {"CropWheat"}
]
print("\nAFTER REMOVING SoilDry")
print("-" * 40)
result_apply = resolution_refutation(

```

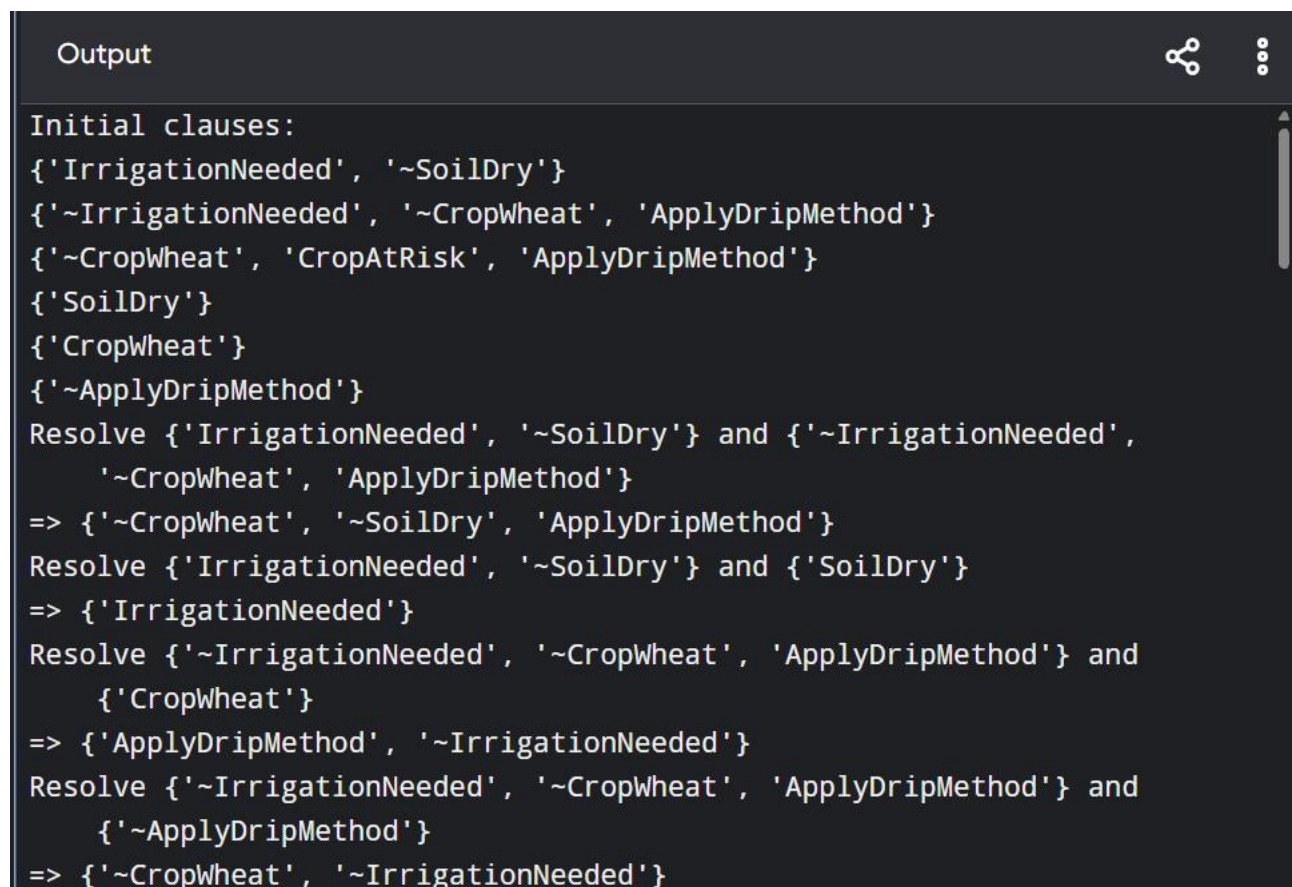


```

    kb_without_soil,
    "ApplyDripMethod"
)
result_risk = resolution_refutation(
    kb_without_soil,
    "CropAtRisk"
)
print("\nApplyDripMethod provable:", result_apply)
print("CropAtRisk provable:", result_risk)

```

OUTPUT :



```

Output
Initial clauses:
{'IrrigationNeeded', '~SoilDry'}
{'~IrrigationNeeded', '~CropWheat', 'ApplyDripMethod'}
{'~CropWheat', 'CropAtRisk', 'ApplyDripMethod'}
{'SoilDry'}
{'CropWheat'}
{'~ApplyDripMethod'}
Resolve {'IrrigationNeeded', '~SoilDry'} and {'~IrrigationNeeded',
 '~CropWheat', 'ApplyDripMethod'}
=> {'~CropWheat', '~SoilDry', 'ApplyDripMethod'}
Resolve {'IrrigationNeeded', '~SoilDry'} and {'SoilDry'}
=> {'IrrigationNeeded'}
Resolve {'~IrrigationNeeded', '~CropWheat', 'ApplyDripMethod'} and
 {'CropWheat'}
=> {'ApplyDripMethod', '~IrrigationNeeded'}
Resolve {'~IrrigationNeeded', '~CropWheat', 'ApplyDripMethod'} and
 {'~ApplyDripMethod'}
=> {'~CropWheat', '~IrrigationNeeded'}

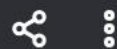
```

Output



```
Resolve {'~CropWheat', 'CropAtRisk', 'ApplyDripMethod'} and {'CropWheat'}
=> {'CropAtRisk', 'ApplyDripMethod'}
Resolve {'~CropWheat', 'CropAtRisk', 'ApplyDripMethod'} and
      {'~ApplyDripMethod'}
=> {'~CropWheat', 'CropAtRisk'}
Resolve {'IrrigationNeeded', '~SoilDry'} and {'ApplyDripMethod',
      '~IrrigationNeeded'}
=> {'~SoilDry', 'ApplyDripMethod'}
Resolve {'IrrigationNeeded', '~SoilDry'} and {'~CropWheat',
      '~IrrigationNeeded'}
=> {'~CropWheat', '~SoilDry'}
Resolve {'~IrrigationNeeded', '~CropWheat', 'ApplyDripMethod'} and
      {'IrrigationNeeded'}
=> {'~CropWheat', 'ApplyDripMethod'}
Resolve {'CropWheat'} and {'~CropWheat', '~IrrigationNeeded'}
=> {'~IrrigationNeeded'}
Resolve {'CropWheat'} and {'~CropWheat', 'CropAtRisk'}
=> {'CropAtRisk'}
```

Output



Derived empty clause:

□

Theorem: ApplyDripMethod

Proved: True

AFTER REMOVING SoilDry

Initial clauses:

{ 'IrrigationNeeded', '~SoilDry' }

{ '~IrrigationNeeded', '~CropWheat', 'ApplyDripMethod' }

{ '~CropWheat', 'CropAtRisk', 'ApplyDripMethod' }

{ 'CropWheat' }

{ '~ApplyDripMethod' }

Resolve { 'IrrigationNeeded', '~SoilDry' } and { '~IrrigationNeeded',
 '~CropWheat', 'ApplyDripMethod' }

=> { '~CropWheat', '~SoilDry', 'ApplyDripMethod' }

Resolve { '~IrrigationNeeded', '~CropWheat', 'ApplyDripMethod' } and

Output



```
=> {'~CropWheat', '~SoilDry', 'ApplyDripMethod'}
Resolve {'~IrrigationNeeded', '~CropWheat', 'ApplyDripMethod'} and
      {'CropWheat'}
=> {'ApplyDripMethod', '~IrrigationNeeded'}
Resolve {'~CropWheat', 'CropAtRisk', 'ApplyDripMethod'} and {'CropWheat'}
=> {'CropAtRisk', 'ApplyDripMethod'}
Resolve {'~CropWheat', 'CropAtRisk', 'ApplyDripMethod'} and
      {'~CropAtRisk'}
=> {'~CropWheat', 'ApplyDripMethod'}
Resolve {'IrrigationNeeded', '~SoilDry'} and {'ApplyDripMethod',
      '~IrrigationNeeded'}
=> {'~SoilDry', 'ApplyDripMethod'}
Resolve {'CropWheat'} and {'~CropWheat', 'ApplyDripMethod'}
=> {'ApplyDripMethod'}

ApplyDripMethod provable: False
CropAtRisk provable: False
```

CASE STUDY 4 — Wumpus World Knowledge Agent:

(a) Belief State and Modus Ponens:

Percepts

The agent perceives : Stench [1,2] Breeze [1,1] Glitter [2,2]

Therefore the initial belief state is:

KB={Stench[1,2], Breeze[1,1], Glitter[2,2]}

Rule 5

Stench[1,2] $\rightarrow$ WumpusAdjacent[1,2]

Since : Stench[1,2] is true,

by Modus Ponens : WumpusAdjacent [1,2]

Rule 6

Breeze [1,1] $\rightarrow$ PitAdjacent [1,1]

Since:

Breeze[1,1] is true : PitAdjacent[1,1]

Rule 7

Glitter[x,y] $\rightarrow$ Gold [x,y]

For [2,2]:

Glitter[2,2] $\rightarrow$ Gold[2,2]

Therefore : Gold[2,2]

Rule 8

$$\neg \text{Wumpus}[x,y] \wedge \neg \text{Pit}[x,y] \rightarrow \text{Safe}[x,y]$$

There are no explicit negative facts such as:

$$\neg \text{Wumpus}[1,1]$$

or:

$$\neg \text{Pit}[1,1]$$

Therefore, Rule 8 cannot currently derive any Safe cells.

Derived knowledge

WumpusAdjacent [1,2]

PitAdjacent [1,1]

Gold[2,2]

(b) Possible Wumpus and Pit Locations:

Assuming the standard Wumpus World adjacency rule, cells adjacent to [1,2] are:

[1,1], [1,3], [2,2]

Therefore, the Wumpus could be in:

[1,1], [1,3], [2,2]

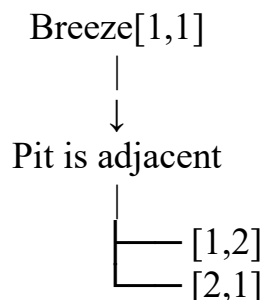
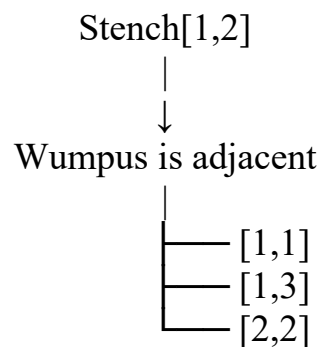
The Breeze at [1,1] means a pit can be in adjacent cells:

[1,2], [2,1]

Therefore:

Pit possibilities = [1,2], [2,1]

Reasoning :



The important point is that a stench does not tell us the exact Wumpus cell. It gives a set of possible locations.

(c) Is the Path Safe?

Proposed path : $[1,1] \rightarrow [1,2] \rightarrow [2,2]$

The agent wants to collect gold at $[2,2]$.

We know , Gold $[2,2]$ because of the glitter percept.

However : Cell $[1,2]$

From , Breeze $[1,1]$ we know that a pit may be at $[1,2]$.

Also, $[1,2]$ is a neighboring cell of the stench location $[1,2]$? More precisely, the stench at $[1,2]$ tells us the Wumpus is adjacent to $[1,2]$, so it does not directly prove that $[1,2]$ itself contains the Wumpus.

Nevertheless, the path cannot be proven safe because $[1,2]$ may contain a pit.

Cell $[2,2]$

From : Stench $[1,2]$ the Wumpus could potentially be at $[2,2]$

Therefore $[2,2]$ also cannot be confirmed safe.

Final conclusion :

The path $[1,1] \rightarrow [1,2] \rightarrow [2,2]$

is not logically guaranteed to be safe.

Path is unsafe/uncertain

The correct AI-agent decision should be to avoid the path until additional information is obtained.

PYTHON CODE :

```
facts = {
    ("Stench", 1, 2),
    ("Breeze", 1, 1),
    ("Glitter", 2, 2)
}
def adjacent(cell1, cell2):
    r1, c1 = cell1
    r2, c2 = cell2

    return abs(r1 - r2) + abs(c1 - c2) == 1
grid = [
    (r, c)
    for r in range(1, 5)
    for c in range(1, 5)
]
derived = set(facts)
changed = True
while changed:
    changed = False
    stench_cells = [
        (x, y)
        for predicate, x, y in derived
```

```

    if predicate == "Stench"
]
for stench_cell in stench_cells:
    possible_wumpus = [
        cell
        for cell in grid
        if adjacent(stench_cell, cell)
    ]
    for cell in possible_wumpus:
        fact = ("PossibleWumpus", cell[0], cell[1])
        if fact not in derived:
            derived.add(fact)
            changed = True
breeze_cells = [
    (x, y)
    for predicate, x, y in derived
    if predicate == "Breeze"
]
for breeze_cell in breeze_cells:
    possible_pits = [
        cell
        for cell in grid
        if adjacent(breeze_cell, cell)
    ]
    for cell in possible_pits:
        fact = ("PossiblePit", cell[0], cell[1])
        if fact not in derived:
            derived.add(fact)
            changed = True
glitter_cells = [
    (x, y)
    for predicate, x, y in derived
    if predicate == "Glitter"
]
for x, y in glitter_cells:
    fact = ("Gold", x, y)
    if fact not in derived:
        derived.add(fact)
        changed = True
print("DERIVED KNOWLEDGE")
print("-" * 40)
for fact in sorted(derived, key=str):
    print(fact)
wumpus_locations = {
    (x, y)

```

```

    for predicate, x, y in derived
    if predicate == "PossibleWumpus"
}
pit_locations = {
    (x, y)
    for predicate, x, y in derived
    if predicate == "PossiblePit"
}
gold_locations = {
    (x, y)
    for predicate, x, y in derived
    if predicate == "Gold"
}
print("\nPossible Wumpus locations:")
print(sorted(wumpus_locations))
print("\nPossible Pit locations:")
print(sorted(pit_locations))
print("\nGold location:")
print(sorted(gold_locations))
path = [
    (1, 1),
    (1, 2),
    (2, 2)
]
print("\nPATH SAFETY ANALYSIS")
print("-" * 40)
safe = True
for cell in path:
    wumpus_possible = cell in wumpus_locations
    pit_possible = cell in pit_locations
    print(f"Cell {cell}:")
    if wumpus_possible:
        print(" Wumpus may be present.")
    if pit_possible:
        print(" Pit may be present.")
    if wumpus_possible or pit_possible:
        print(" NOT PROVEN SAFE")
        safe = False
    else:
        print(" No danger inferred from current knowledge.")
print("\nFinal decision:")
if safe:
    print("The complete path is logically safe.")
else:
    print("The path is NOT guaranteed safe.")

```



```
print("Agent should avoid it until more information is obtained.")
```

OUTPUT :

Output

DERIVED KNOWLEDGE

```
('Breeze', 1, 1)
('Glitter', 2, 2)
('Gold', 2, 2)
('PossiblePit', 1, 2)
('PossiblePit', 2, 1)
('PossibleWumpus', 1, 1)
('PossibleWumpus', 1, 3)
('PossibleWumpus', 2, 2)
('Stench', 1, 2)
```

Possible Wumpus locations:

```
[(1, 1), (1, 3), (2, 2)]
```

Output

Possible Pit locations:

```
[(1, 2), (2, 1)]
```

Gold location:

```
[(2, 2)]
```

PATH SAFETY ANALYSIS

Cell (1, 1):

Wumpus may be present.

NOT PROVEN SAFE

Cell (1, 2):

Pit may be present.

NOT PROVEN SAFE

Output

Cell (1, 1):

Wumpus may be present.

NOT PROVEN SAFE

Cell (1, 2):

Pit may be present.

NOT PROVEN SAFE

Cell (2, 2):

Wumpus may be present.

NOT PROVEN SAFE

Final decision:

The path is NOT guaranteed safe.

Agent should avoid it until more information is obtained.

=== Code Execution Successful ===